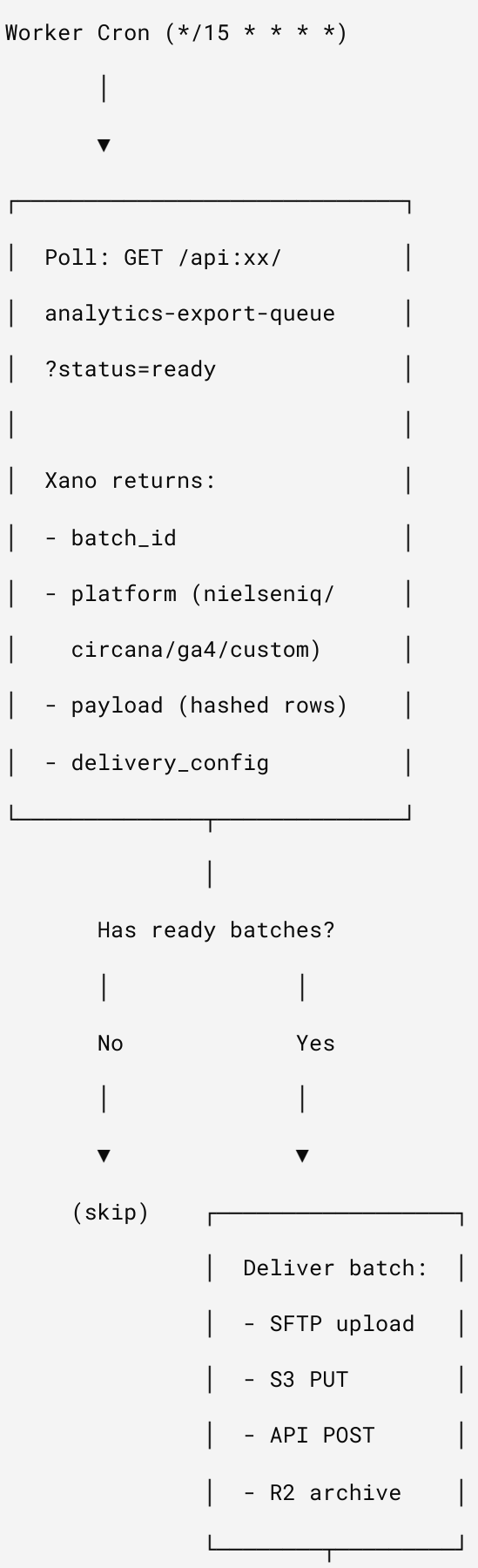
SPECIFICATION

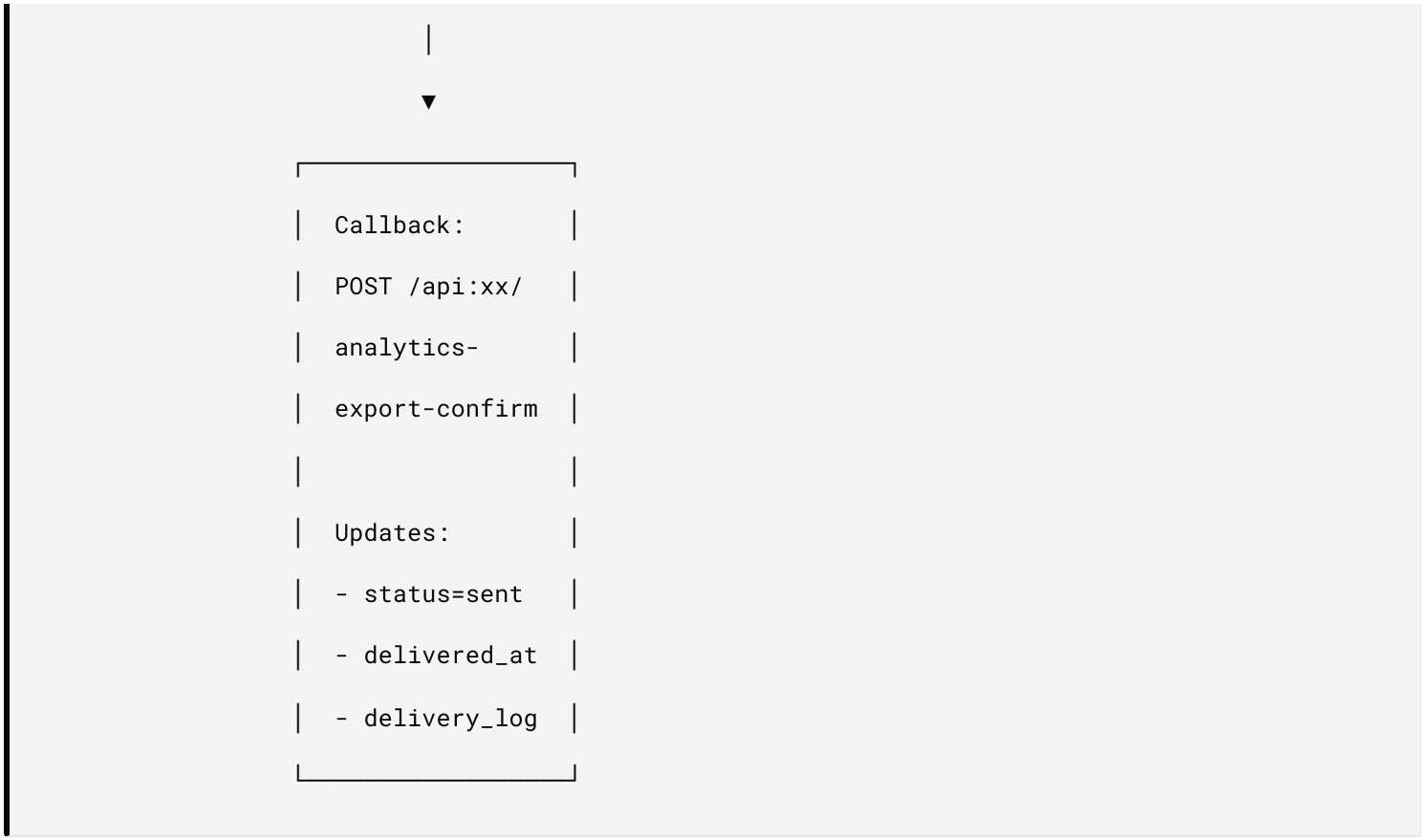
Analytics Export via Xano Polling + Worker Cron

Version 1.0 · 2026-05-27 · Specification. The analytics export rail — **Xano polling + Worker cron** — with the architecture, queue tables, and export contract for downstream analytics consumers.

Version: **1.0** Date: **2026-05-27** Status: **Specification**

ARCHITECTURE




Callback:
POST /api:xx/
analytics-
export-confirm
Updates:
- status=sent
- delivered_at
- delivery_log

XANO SIDE: EXPORT QUEUE TABLE

Table: `analytics_export_queue` (new)

COLUMN	TYPE	DESCRIPTION
`id`	int (auto)	Primary key
`batch_id`	text (unique)	UUID — `aex_{uuid}`
`platform`	enum	`nielseniq`, `circana`, `ga4_bulk`, `adobe_bulk`, `custom`
`status`	enum	`pending`, `ready`, `processing`, `sent`, `failed`, `expired`
`tenant_shop`	text	Shop domain (multi-tenant scoping)
`row_count`	int	Number of records in batch
`payload`	json	Pre-hashed, pre-formatted export data
`delivery_config`	json	`{ method, endpoint, credentials_key }`
`filters`	json	`{ date_from, date_to, segments, consent_scope }`
`created_at`	timestamp	When batch was queued
`ready_at`	timestamp	When Xano finished preparing
`delivered_at`	timestamp	When worker confirmed delivery
`delivery_log`	json	Response from analytics platform
`expires_at`	timestamp	Auto-expire undelivered batches (7 days)

Xano Background Task: `prepare_analytics_export`

Trigger: Scheduled (daily at 02:00 UTC) or on-demand via API

Logic:

1. Query `storefront_users` + `user_claims` + `consent_records`

2. Filter: only users with `analytics_sharing_consent = true`
3. Hash PII: `SHA-256(lowercase(email))` — same as Adobe AEP pattern
4. Join Shopify order data (SKUs → UPCs via lookup table)
5. Format per platform spec (NielsenIQ CSV vs Circana JSON)
6. Insert into `analytics_export_queue` with `status = ready`

Xano API Endpoints (new)

`GET /API:{GROUP}/ANALYTICS-EXPORT-QUEUE`

```
Query params:
  status=ready          (filter by status)
  platform=nielseniq    (optional platform filter)
  shop=store.myshopify.com  (tenant scoping)

Response:
{
  "items": [
    {
      "batch_id": "aex_a1b2c3d4",
      "platform": "nielseniq",
      "status": "ready",
      "row_count": 1247,
      "payload": [ ... ],  // pre-formatted rows
      "delivery_config": {
        "method": "sftp",
        "host": "sftp.nielseniq.com",
        "path": "/incoming/brand-xyz/",
        "credentials_key": "NIELSENIQ_SFTP_KEY"
      },
      "ready_at": "2026-05-27T02:15:00Z"
    }
  ]
}
```

`POST /API:{GROUP}/ANALYTICS-EXPORT-CONFIRM`

Body:

```
{
  "batch_id": "aex_a1b2c3d4",
  "status": "sent",          // or "failed"
  "delivery_log": {
    "http_status": 200,
    "bytes_sent": 145230,
    "remote_ack": "accepted"
  }
}
```

Addition to `scheduled()` handler

```
// ── Analytics Export Polling ──
// Runs on existing */15 cron. Xano prepares batches on its own schedule.
// Worker polls for ready batches and handles delivery.

async function pollAndDeliverAnalyticsExports(cfg: ResolvedConfig, env: Env, shop: string): Prc
  // 1. Poll Xano for ready batches
  const queueUrl = `${cfg.xanoBaseUrl}/analytics-export-queue?status=ready&shop=${encodeURIComp
  const res = await fetch(queueUrl, {
    headers: { Authorization: `Bearer ${cfg.xanoApiKey}` },
  });
  if (!res.ok) return;

  const data = await res.json() as { items: AnalyticsExportBatch[] };
  if (!data.items?.length) return;

  for (const batch of data.items) {
    try {
      // 2. Mark as processing (prevent double-delivery)
      await confirmExportStatus(cfg, batch.batch_id, "processing", {});

      // 3. Deliver based on method
      const result = await deliverBatch(cfg, env, batch);

      // 4. Confirm delivery to Xano
      await confirmExportStatus(cfg, batch.batch_id, result.ok ? "sent" : "failed", result.log)
```

```
      console.log(`Analytics export ${batch.batch_id} → ${batch.platform}: ${result.ok ? "deliv
    } catch (e) {
      await confirmExportStatus(cfg, batch.batch_id, "failed", {
        error: e instanceof Error ? e.message : String(e),
      });
    }
  }
}

async function deliverBatch(cfg: ResolvedConfig, env: Env, batch: AnalyticsExportBatch): Promis
  const { method } = batch.delivery_config;

  switch (method) {
    case "sftp":
      // Stage to R2, then trigger SFTP bridge
      return await deliverViaSFTP(cfg, env, batch);

    case "s3":
      // Direct S3 PUT via presigned URL
      return await deliverViaS3(cfg, env, batch);

    case "api":
      // Direct HTTP POST to analytics platform API
      return await deliverViaAPI(cfg, env, batch);

    case "r2":
      // Archive to R2 for manual pickup
      return await deliverViaR2(env, batch);

    default:
```

```
      return { ok: false, log: { error: `Unknown delivery method: ${method}` } };
  }
}
```

Delivery Methods

SFTP (NIELSENIQ STANDARD)

```
Worker → R2 (stage CSV) → SFTP Bridge (Cloudflare Worker or external)
                          → sftp.nielseniq.com/incoming/
```

> Note: Cloudflare Workers can't do raw SFTP. Options:
>
> 1. R2 + external SFTP bridge (Lambda/EC2 with cron)
> 2. NielsenIQ S3 drop alternative (preferred if available)
> 3. Third-party service (e.g., Stitch, Fivetran connector)

S3 DROP (CIRCANA / MODERN PLATFORMS)

```
// Presigned URL from Xano delivery_config
const presignedUrl = batch.delivery_config.presigned_url;
await fetch(presignedUrl, {
  method: "PUT",
  headers: { "Content-Type": "text/csv" },
  body: formatCSV(batch.payload),
});
```

API POST (GA4 BULK / CUSTOM ENDPOINTS)

```
// Direct HTTP POST — same pattern as pushToGA4
await fetch(batch.delivery_config.endpoint, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": `Bearer ${batch.delivery_config.api_key}`,
  },
  body: JSON.stringify(batch.payload),
});
```

R2 ARCHIVE (MANUAL PICKUP / AUDIT TRAIL)

```
// Always archive — even when delivering elsewhere
await env.ANALYTICS_EXPORTS.put(
  `exports/${batch.platform}/${batch.batch_id}.json`,
  JSON.stringify(batch.payload),
  { customMetadata: { shop: batch.tenant_shop, rows: String(batch.row_count) } }
);
```

EXPORT PAYLOAD FORMAT

NielsenIQ Format (CSV)

```
hashed_email,order_date,upc,quantity,revenue,channel,segment,consent_scope
a1b2c3...,2026-05-20,012345678901,2,29.98,dtc_shopify,health_conscious,analytics_sharing
d4e5f6...,2026-05-21,012345678902,1,14.99,dtc_shopify,active_lifestyle,analytics_sharing
```

Circana Format (JSON)

```
{
  "export_id": "aex_a1b2c3d4",
  "brand": "OMEN",
  "export_date": "2026-05-27",
  "records": [
    {
      "hashed_id": "sha256:a1b2c3...",
      "transactions": [
        { "upc": "012345678901", "qty": 2, "revenue": 29.98, "date": "2026-05-20" }
      ],
      "segments": ["health_conscious", "subscriber"],
      "channel": "dtc_shopify"
    }
  ]
}
```

PII Hashing (matches existing Adobe AEP pattern)

```
// Already in worker — reuse for analytics export
async function hashPII(value: string): Promise<string> {
  const hash = await crypto.subtle.digest(
    "SHA-256",
    new TextEncoder().encode(value.toLowerCase().trim())
  );
  return Array.from(new Uint8Array(hash)).map(b => b.toString(16).padStart(2, "0")).join("");
}
```

CONSENT GATE

New consent scope: `analytics_partners`

Add to `consent_records` table in Xano:

FIELD	VALUE
`scope`	`analytics_partners`
`granted_at`	timestamp
`method`	`explicit_optin`
`partner_list`	`["nielseniq", "circana"]`

Consent check in Xano background task:

```
-- Only export users who have analytics_partners consent
SELECT u.id, SHA256(LOWER(u.email)) as hashed_email, ...
FROM storefront_users u
JOIN consent_records c ON c.user_id = u.id
WHERE c.scope = 'analytics_partners'
  AND c.revoked_at IS NULL
```

Extension UI: consent toggle

Add to Auth tab alongside existing A2A/AP2 toggles:

```
Analytics Partner Sharing
 Share hashed purchase data with measurement partners
 (NielsenIQ, Circana) for market research.
```

CRON SCHEDULE MATRIX

TASK	SCHEDULE	HANDLER
Customer sync (Shopify → Xano → Webflow)	`*/15 * * * *`	Existing `syncCustomers()`
Shopify token refresh	`*/15 * * * *`	Existing `refreshShopifyTokenIfNeeded()`
Analytics export poll	`*/15 * * * *`	**New** **`pollAndDeliverAnalyticsExports()`**
Analytics batch preparation (Xano-side)	`0 2 * * *` (daily 2am)	Xano background task

> The worker cron runs every 15 minutes. Analytics batches are prepared daily by Xano. On the first cron run after Xano marks a batch `ready`, the worker picks it up and delivers. Worst-case delivery latency: 15 minutes after Xano finishes preparation.

CONFIG: NEW KV/ENV FIELDS

wrangler.toml additions

```
# R2 bucket for analytics export archives

[[r2_buckets]]

binding = "ANALYTICS_EXPORTS"

bucket_name = "crm-analytics-exports"
```

Per-tenant config (KV)

```
{
  "analytics_export_enabled": true,
  "analytics_platforms": ["nielseniq", "circana"],
  "analytics_consent_scope": "analytics_partners",
  "analytics_upc_mapping_table": 190
}
```

Secrets (per platform)

```
NIELSENIQ_SFTP_KEY        (SFTP private key or password)
NIELSENIQ_S3_ACCESS_KEY   (if using S3 drop instead)
CIRCANA_API_KEY            (Unify platform API key)
```

STAKEHOLDER MATRIX

CAPABILITY	A (CREATOR)	B (SHARED)	C (PRIVATE)
Enable analytics export	✓ Platform config	×	✓ Own config
Set platform credentials	✓ `wrangler secret put`	×	✓ Own secrets
View export history	✓ `/admin/analytics-exports`	×	✓ Own endpoint
Manage consent toggles	✓ Extension Auth tab	✓ Extension Auth tab	✓ Extension Auth tab
UPC mapping table	✓ Xano admin	×	✓ Own Xano
Custom export formats	✓ Xano task config	×	✓ Own Xano

IMPLEMENTATION PHASES

Phase 1: Infrastructure (1-2 days)

- ☐ Create `analytics_export_queue` table in Xano
- ☐ Create R2 bucket `crm-analytics-exports`
- ☐ Add `analytics_partners` consent scope
- ☐ Add consent toggle to extension Auth tab

Phase 2: Xano Export Preparation (2-3 days)

- ☐ Build Xano background task: query → hash → format → queue
- ☐ Create UPC mapping table (Shopify SKU → NielsenIQ UPC)
- ☐ Build Xano API endpoints: `analytics-export-queue`, `analytics-export-confirm`
- ☐ Test with mock platform credentials

Phase 3: Worker Delivery (1-2 days)

- ☐ Add `pollAndDeliverAnalyticsExports()` to cron handler
- ☐ Implement R2 archive delivery (always-on audit trail)
- ☐ Implement API POST delivery (for GA4 bulk / custom)
- ☐ Implement S3 delivery (for NielsenIQ/Circana drop)

Phase 4: Platform Onboarding (external dependency)

- ☐ NielsenIQ Connect contract + SFTP credentials
- ☐ Circana Unify contract + API access
- ☐ Test end-to-end with real platform endpoints
- ☐ Verify hashed ID match rates